PYTHON

Module 8: Data Types & Categories

Understanding Data Types

Data types are the classifications we give to different kinds of data in programming. It tells the computer how to store information and what operations can be performed on it.

In Python, everything is an object, and every object has a data type. Understanding these is the key to mastering data manipulation in the CodeHive ecosystem.

Functional Overview:

- **Performance: Different types occupy different amounts of memory.**
- **Logic: You can add two integers, but you cannot 'add' a list and a dictionary without specific methods.**

Text & Numeric Types

These are the most commonly used data types in any Python application.

• String (`str ): For text data. Example:  x = "CodeHive"`

`• Integer ( int ): Whole numbers without decimals. Example:  x = 100`

`• Floating Point ( float ): Numbers containing decimals. Example:  x = 99.99`

`• Complex ( complex ): Used for mathematical engineering. Example:  x = 3+5j`

Sequence Types

Sequences allow you to store multiple items in a single variable.

- **List (`list`): Ordered and changeable. Example: `["apple", "banana"]`**
- **Tuple (`tuple`): Ordered but unchangeable. Example: `("apple", "banana")`**
- **Range (`range`): Used primarily for loops and generating number sequences. Example: `range(10)`**

Mapping & Set Types

For storing organized or unique data collections.

- **Dictionary (`dict`): Stores data in Key-Value pairs. Example: `{"name": "John", "age": 30}`**
- **Set (`set`): Unordered collection of unique items. No duplicates allowed!**
- **Frozenset (`frozenset`): An immutable (unchangeable) version of a set.**

Boolean & None Types

Used for logic and representing empty states.

- **Boolean (`bool`): Represents `True` or `False`. Values result from comparison operations like `10 > 9`.**

- **NoneType (`None`): Represents the absence of a value or a null pointer. Useful for initializing variables that will be filled later.**

Binary & Memory Types

For low-level data processing and file handling.

- **Bytes (`bytes`): Immutable sequences of single bytes.**
- **Bytearray (`bytearray`): A mutable version of bytes.**
- **Memoryview (`memoryview`): Allows Python code to access the internal data of an object that supports the buffer protocol without copying.**

Checking & Setting Types

Python is dynamically typed, but sometimes you need to enforce a specific type or verify one.

- **Checking: Use `type(x)` to see the current classification.**
- **Constructor Functions: You can explicitly set a type using functions like `list()`, `dict()`, or `str()` to wrap your data.**

Example: `x = set(("A", "B", "C"))` ensures the collection is a set even if passed as a tuple.

Best Practices: Choosing a Type

At CodeHive, we recommend picking the right 'container' for the job:

- **Tracking items that change? Use a **List**.**
- **Storing coordinates or fixed config? Use a **Tuple**.**
- **Need fast lookups by name? Use a **Dictionary**.**
- **Need to ensure no duplicates? Use a **Set**.**